

HUNDRED DAYS OF CODE

DAY 14

QUESTION: Write a menu-driven C program using switch-case to input a positive integer and Check whether all its digits are unique.

INPUT: - MENU CHOICE

- A POSITIVE INTEGER

OUTPUT: - **Unique** if no digit occurs more than once.

- **Not Unique** if any digit occurs more than once.
- Appropriate messages for invalid menu choices or non-positive integers.

ALGORITHM: 1. Start.

2. Display the menu:

- 1. Check whether digits are unique
- 2. Exit

3. Read the user's choice.

4. Use switch-case to process the choice.

5. If the choice is 1:

- Read a positive integer n .
- If $n \leq 0$, display **Invalid input** and return to the menu.
- Initialize an array `seen[10]` to zero.
- Set `unique = 1`.
- Extract the last digit using $\text{digit} = n \% 10$.
- Check whether `seen[digit]` is already 1.
- If it is 1, set `unique = 0` and stop checking.
- Otherwise, mark `seen[digit] = 1`.
- Remove the last digit using $n = n / 10$.
- Repeat until all digits have been checked.
- If `unique == 1`, display **Unique**.
- Otherwise, display **Not Unique**.

6. If the choice is 2, terminate the program.

7. For any other choice, display **Invalid choice**.

HUNDRED DAYS OF CODE

DAY 14

8. Repeat the menu until the user selects 2.

9. Stop.

PSEUDOCODE:

STAR

REPEAT

Display menu

Display "1. Check whether digits are unique"

Display "2. Exit"

Read choice

SWITCH(choice)

CASE 1:

Read n

IF $n \leq 0$ THEN

Display "Invalid input"

BREAK

END IF

Set all elements of seen[0..9] to 0

unique $\leftarrow 1$

temp $\leftarrow n$

WHILE temp > 0 DO

digit $\leftarrow$ temp MOD 10

HUNDRED DAYS OF CODE

DAY 14

IF seen[digit] = 1 THEN

 unique $\leftarrow$ 0

 BREAK

END IF

seen[digit] $\leftarrow$ 1

temp $\leftarrow$ temp DIV 10

END WHILE

IF unique = 1 THEN

 Display "Unique"

ELSE

 Display "Not Unique"

END IF

BREAK

CASE 2:

 Display "Program terminated"

BREAK

DEFAULT:

 Display "Invalid choice"

END SWITCH

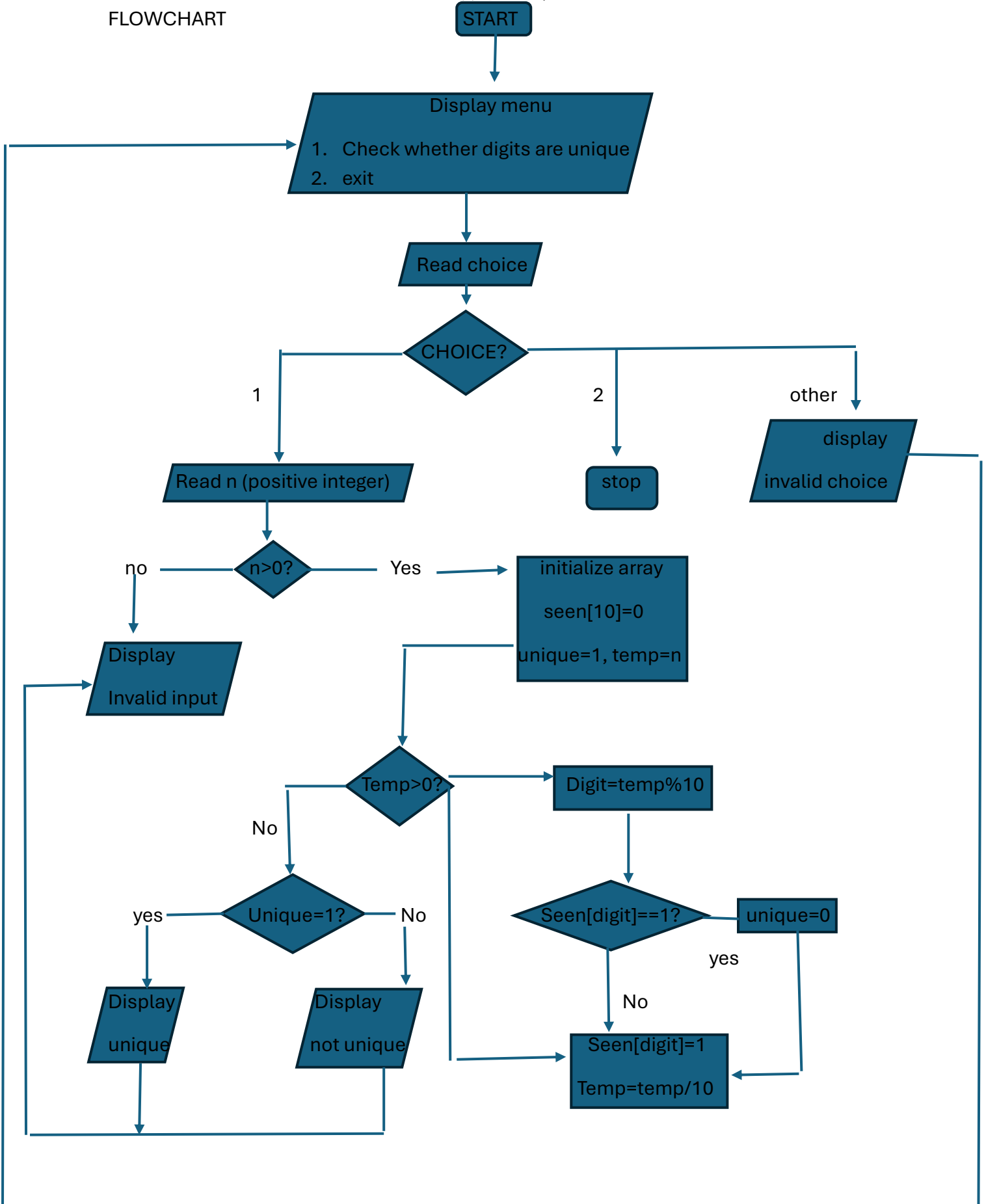
UNTIL choice = 2

STOP

HUNDRED DAYS OF CODE

DAY 14

FLOWCHART



REQUIRED C PROGRAM: #include <stdio.h>

```
int main(void)
{
    int choice, n, digit, temp;

    int seen[10];

    int unique;

    do
    {
        printf("\n--- MENU ---\n");

        printf("1. Check whether digits are unique\n");

        printf("2. Exit\n");

        printf("Enter your choice: ");

        scanf("%d", &choice);

        switch (choice)
        {
            case 1:

                printf("Enter a positive integer: ");

                scanf("%d", &n);

                if (n <= 0)
                {
                    printf("Invalid input. Enter a positive integer.\n");

                    break;
                }
            }
        }
    } while (choice != 2);

    return 0;
}
```

HUNDRED DAYS OF CODE

DAY 14

```
}

/* Initialize the array */
for (int i = 0; i < 10; i++)
    seen[i] = 0;

temp = n;
unique = 1;

/* Check every digit */
while (temp > 0)
{
    digit = temp % 10;

    if (seen[digit] == 1)
    {
        unique = 0;
        break;
    }

    seen[digit] = 1;
    temp = temp / 10;
}

if (unique == 1)
    printf("Output: Unique\n");
else
```

HUNDRED DAYS OF CODE

DAY 14

```
printf("Output: Not Unique\n");

break;

case 2:

printf("Program terminated.\n");

break;

default:

printf("Invalid choice.\n");

}

}while (choice != 2);

return 0;

}
```

TEST CASE TABLE:

TEST CASE	MENU CHOICE	INPUT	EXPECTED OUTPUT	RESULT
1	1	12345	Unique	pass
2	1	12234	Not unique	Pass
3	1	1002	Not unique	Pass

HUNDRED DAYS OF CODE

DAY 14

COMPILATION AND EXECUTION:

```
(kali㉿kali)-[~]  
$ vi unique.c  
  
(kali㉿kali)-[~]  
$ cc unique.c -o unique  
  
(kali㉿kali)-[~]  
$ ./unique  
  
—MENU—  
1. Check whether digits are unique  
2. Exit  
Enter your choice:1  
Enter a positive integer: 12345  
Output: Unique  
  
—MENU—  
1. Check whether digits are unique  
2. Exit  
Enter your choice:1  
Enter a positive integer: 12234  
Output: Not Unique  
  
—MENU—  
1. Check whether digits are unique  
2. Exit  
Enter your choice:1  
Enter a positive integer: 1002  
Output: Not Unique
```
